

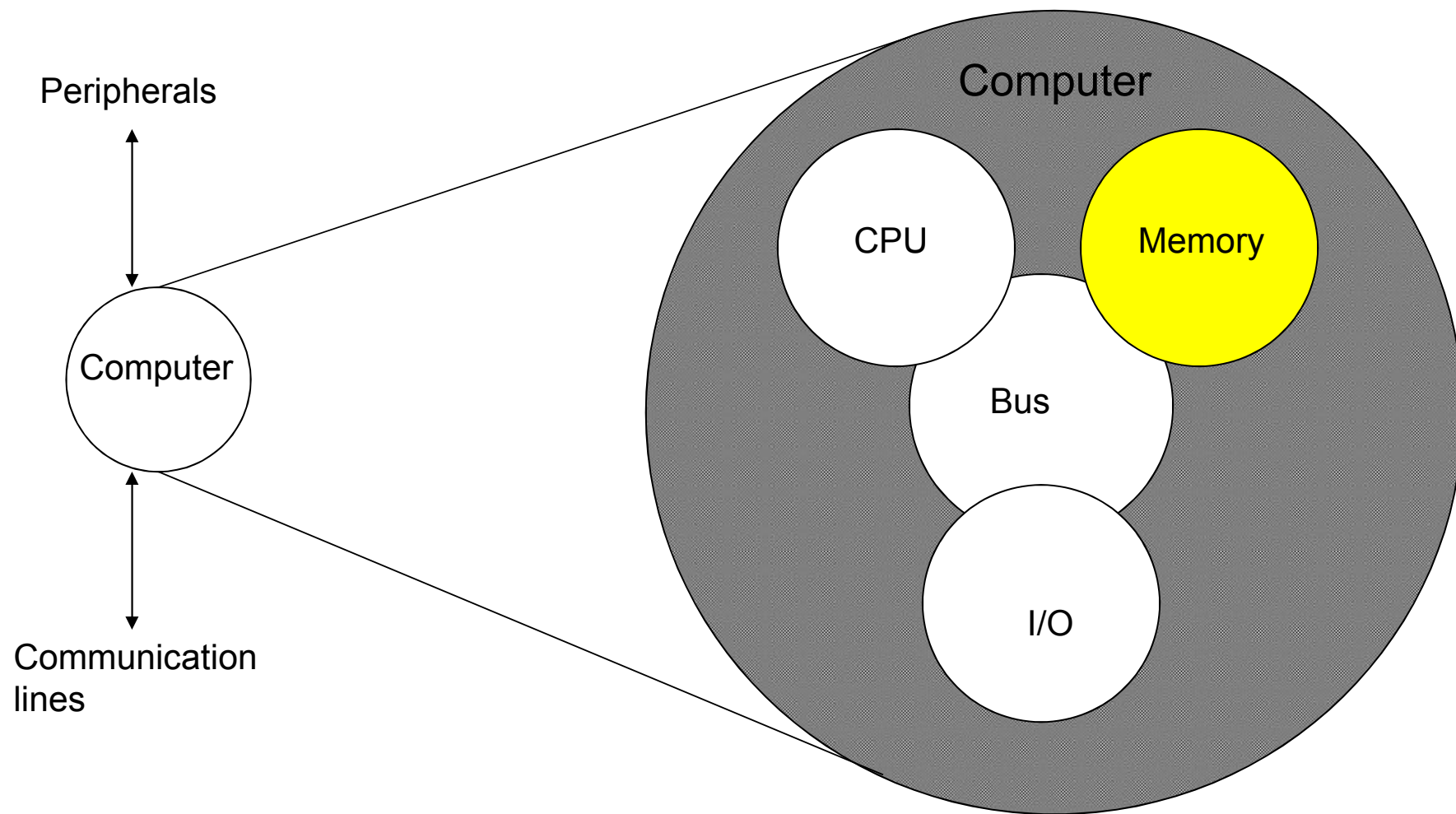
Computer Organization and Architecture (10th Edition) William Stallings

Chapter 4 Cache Memory

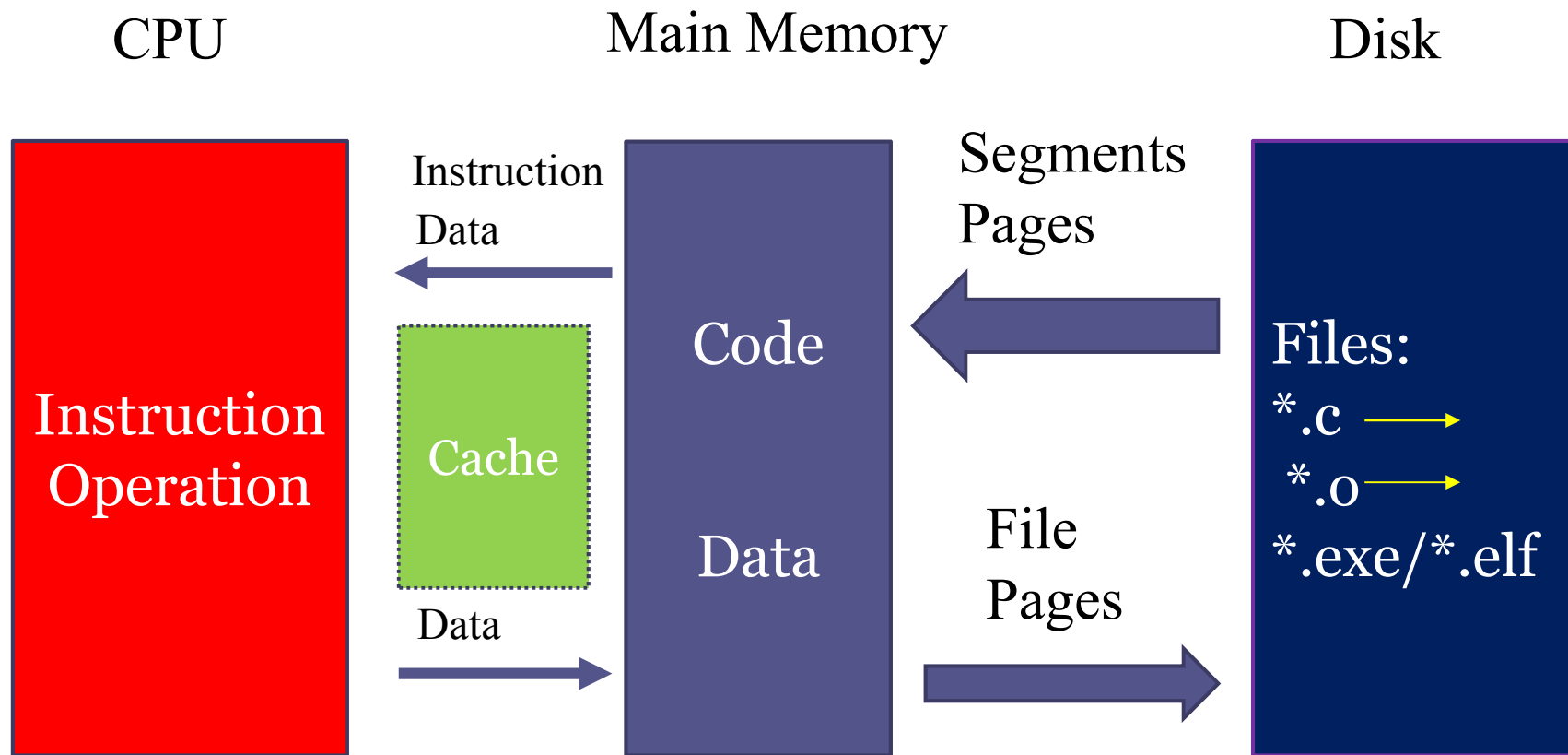
OBJECTIVES:

1. Present an overview of the main characteristics of computer memory systems and the use of a **memory hierarchy**.
2. Describe the basic concepts and intent of **cache memory**.
3. Discuss the key elements of cache design.
4. Distinguish among **direct mapping**, **associative mapping**, and **set-associative mapping**.
5. Understand the performance implications of multiple levels of memory

Structure - Top Level



Computer Processing



Statements, assembly codes, machine codes

```
1  int accum = 0;
2
3  int sum(int x, int y)
4  {
5      int t = x + y;
6      accum += t;
7      return t;
8  }
```

sum:

pushl	%ebp	0:	55
movl	%esp, %ebp	1:	89 e5
movl	12(%ebp), %eax	3:	8b 45 0c
addl	8(%ebp), %eax	6:	03 45 08
addl	%eax, accum	9:	01 05 00 00 00 00
popl	%ebp	f:	5d
ret		10:	c3

55 89 e5 8b 45 0c 03 45 08 01 05 00 00 00 00 5d c3

Memory System

- Computer memory is organized into a hierarchy.
(***processor registers** ← **L1 (L2...) cache** ← **main memory** ← **external memory**) (Memory Organization)
- Decreasing cost/bit, increasing capacity, slower access time (Memory Characteristics)
- **Locality of reference** makes proper designs of cache improve computer performance (Design)



4.1 Computer memory system overview

- Characteristics of memory systems
- The memory hierarchy

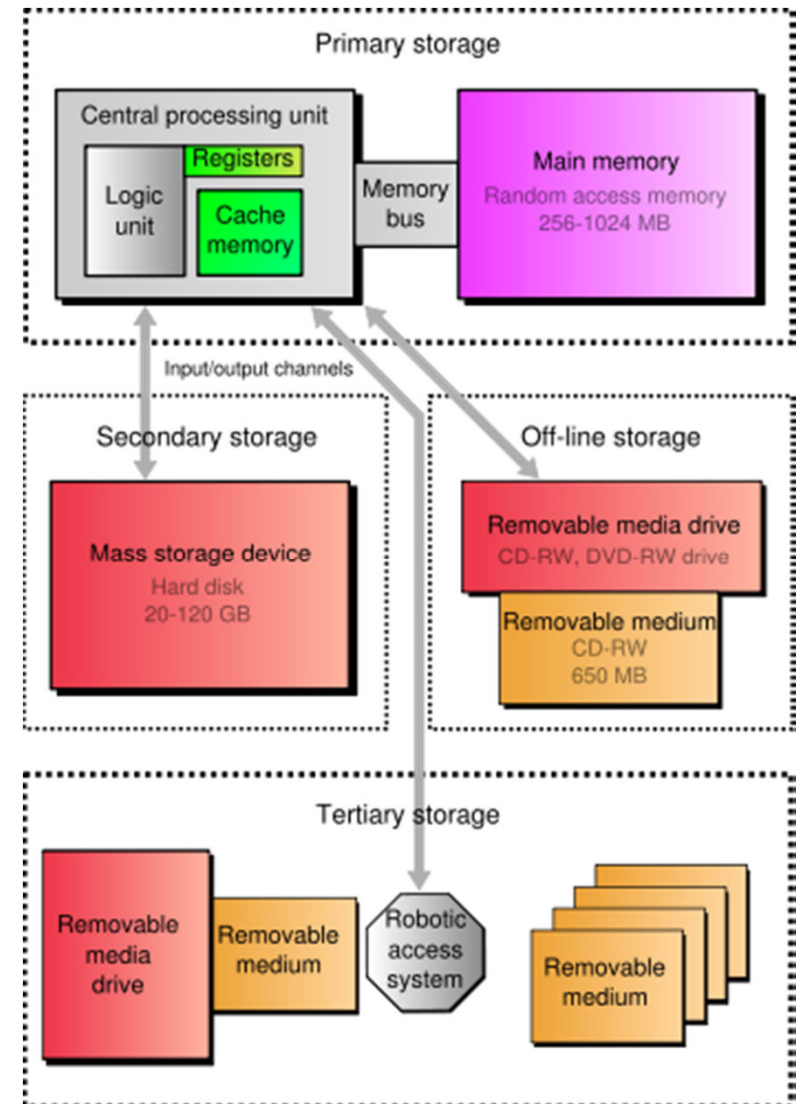


Characteristics

- Location
- Capacity
- Unit of transfer
- Access method
- Performance
- Physical type
- Physical characteristics
- Organization

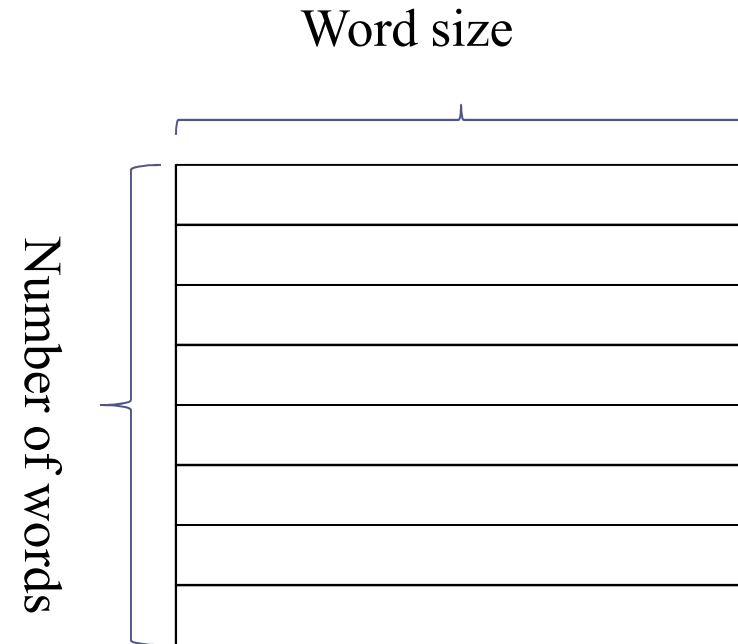
Location

- CPU:
 - Register, Control memory
- Internal
 - Cache, Main memory
- External
 - Disk, Tape, CD, DVD



Capacity

- Word size
 - The natural unit of organisation
 - Intel x86: word size=32bits
- Number of words
 - or Byte



Capacity: $\text{Word size} \times \text{Number of words (or Bytes)}$

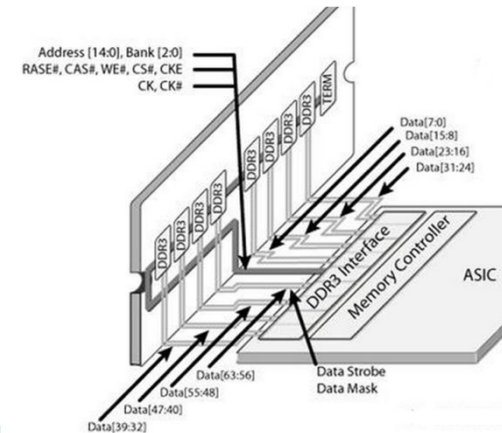
Addressable unit

- Addressable unit

- Smallest location which can be uniquely addressed”
能被唯一寻址的最小存储单位（如内存地址单元）”
- Word or byte
- Address length :A bits
- the number N of addressable units is $2^A = N$
- 32bits, $N=2^{32}$

Unit of Transfer

- Internal
 - Usually governed by data bus width
 - Word internally “在计算机内部（如CPU、总线、寄存器层面），数据以字（Word）为单位进行处理和传输”
- External
 - Usually a **block** which is much larger than a word
 - Cluster disks”在磁盘（如硬盘）中，可寻址单元是簇（Cluster）或块（Block）”



Access Methods (1)

- Sequential

- Start at the beginning and read through in order
- Access time depends on location of data and previous location
- e.g. tape



- Direct

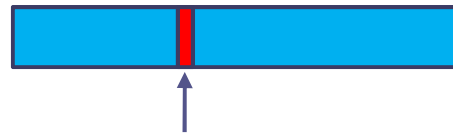
- Individual blocks have unique address
- Access is by jumping to **vicinity** plus **sequential** search
- Access time depends on location and previous location
- e.g. disk



Access Methods (2)

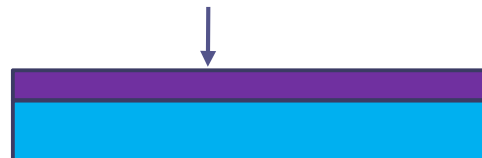
- **Random**

- Individual addresses identify locations exactly
- Access time is independent of location or previous access
- e.g. RAM



- **Associative** ★

- Data is located by a comparison with **contents** of a portion of the store
- Access time is independent of location or previous access
- e.g. cache



Performance

- Access time
 - Time between presenting *the address* and *getting the valid data*
- Memory Cycle time
 - Time may be required for the memory to “*recover*” before next access
 - Cycle time is *access + recovery*
- Transfer Rate R
 - Rate at which data can be moved
 - ① random-access : $R=1/(\text{cycle time})$
 - ② non-random-access:

Performance

$$T_N = T_A + N / R$$

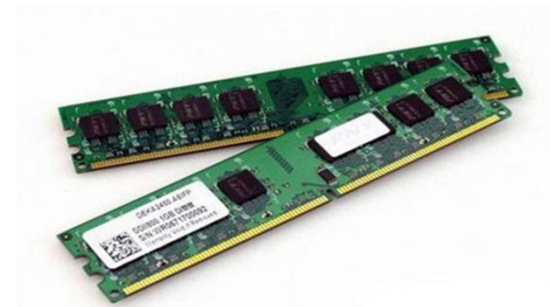
T_n = Average time to read or write n bits

T_A = Average access time

n = Number of bits

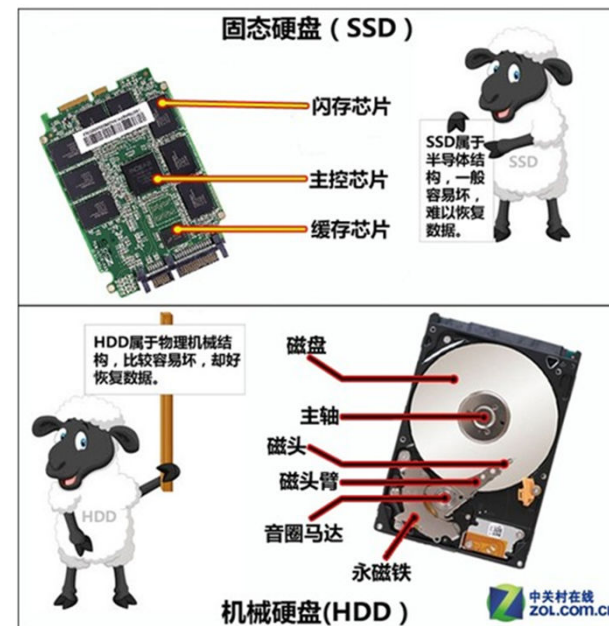
Physical Types

- Semiconductor
 - RAM
- Magnetic
 - Disk & Tape
- Optical
 - CD & DVD
- Magneto-optical



Physical Characteristics

- Decay(即使不断电也会发生,如相变存储器)
- Volatility(断电触发,如Memory; 反之光盘, flash)
- Erasable
- Power consumption



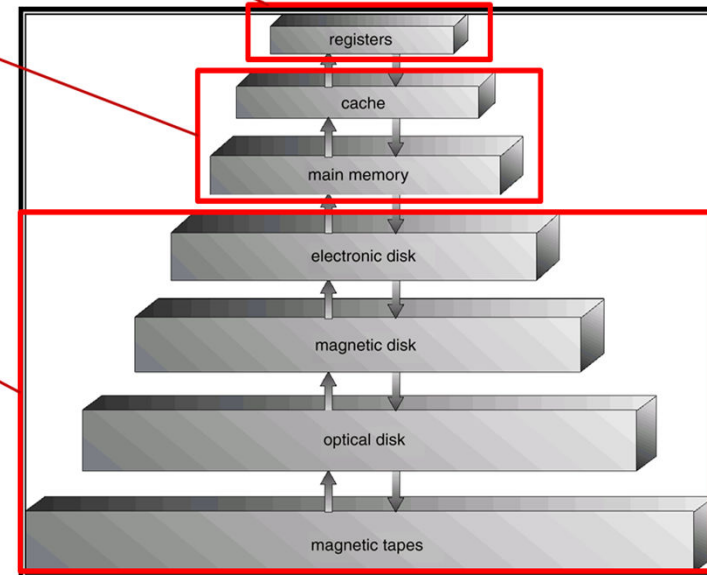
Memory design

- Large enough?
 - Capacity
- How fast?
 - Time is money
- How expensive?
- Dilemma:
 - Faster **access time**, greater cost per bit
 - Greater **capacity**, smaller **cost** per bit
 - Greater capacity, slower access time

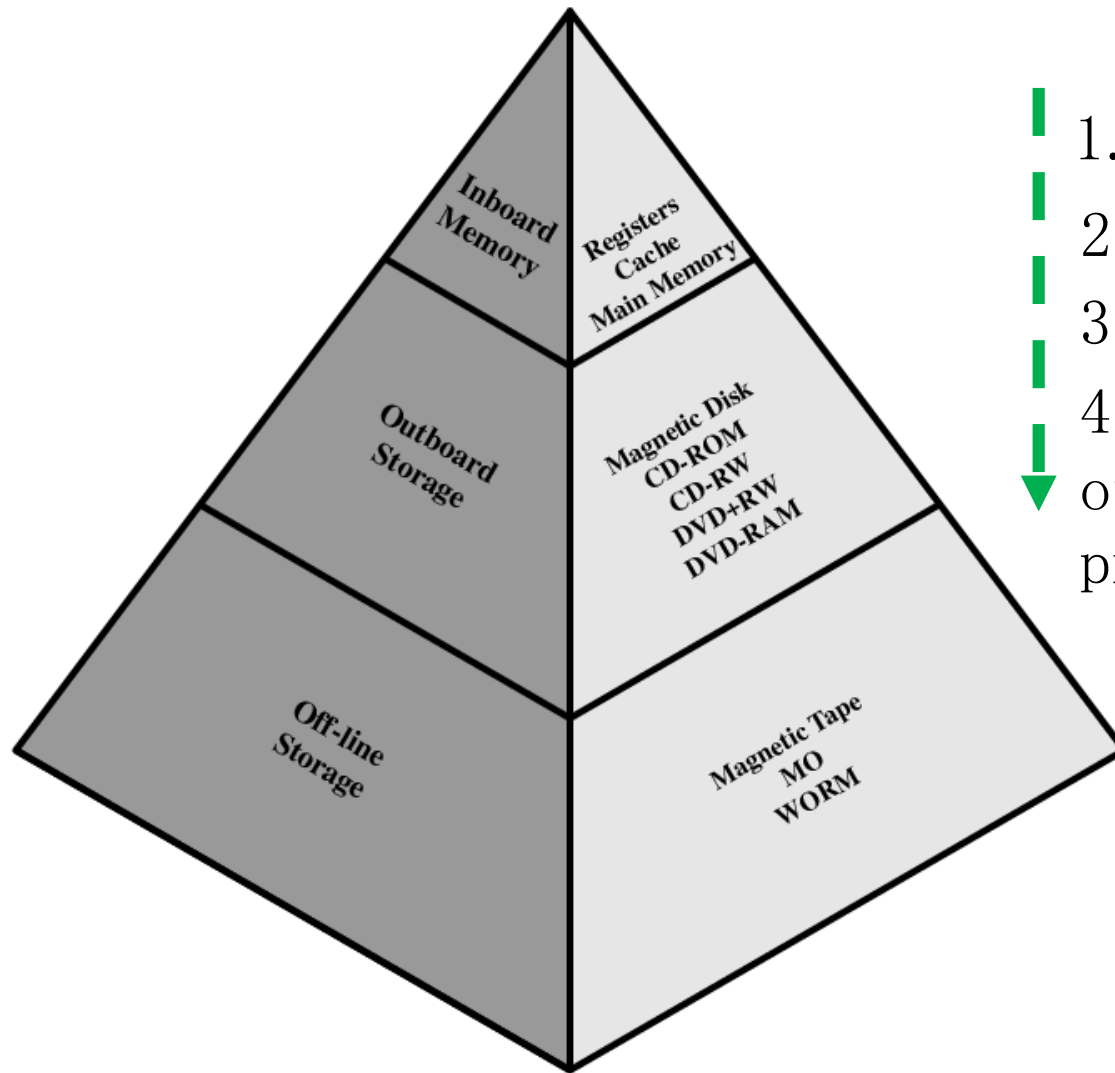


Memory Hierarchy —the way out of the dilemma?

- Registers
 - In CPU
- Internal or Main memory
 - May include one or more levels of cache
 - “RAM”
- External memory
 - Backing store



Memory Hierarchy - Diagram



1. Cost per bit? Decreasing

2. Capacity? Increasing

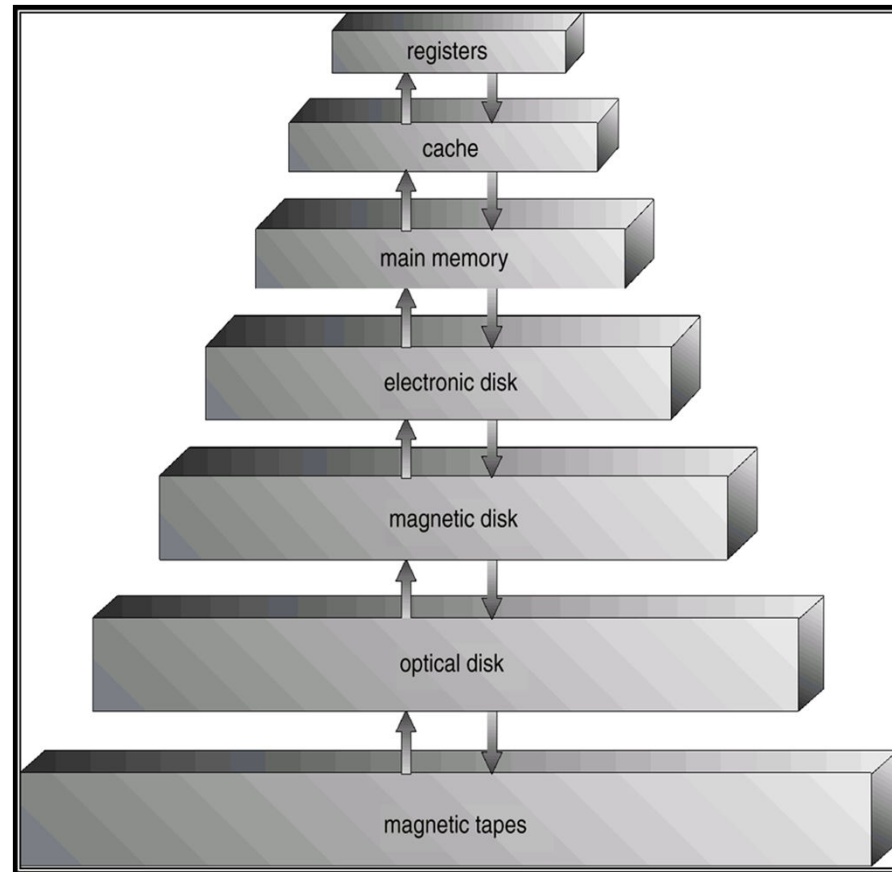
3. Access time? Increasing

4. Frequency of access
of the memory by the
processor?

Decreasing
: Locality of
reference

Hierarchy List

- Registers
- L1 Cache
- L2 Cache
- Main memory
- Disk cache
- Disk
- Optical
- Tape



Performance of two-level memory

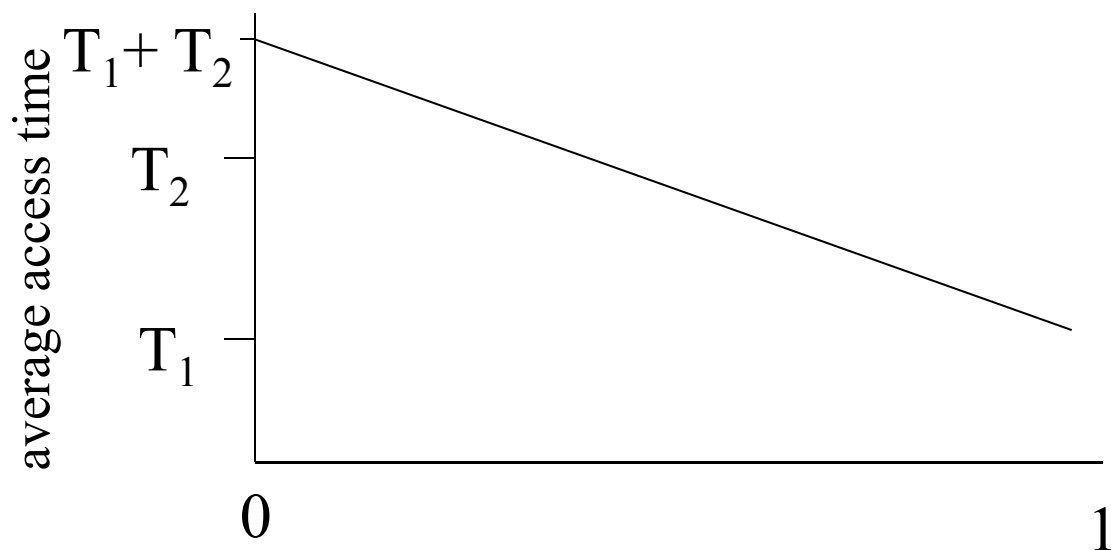
Example 4.1 Suppose that the processor has access to two levels of memory. Level 1 contains 1000 words and has an access time of $0.01 \mu\text{s}$; level 2 contains 100,000 words and has an access time of $0.1 \mu\text{s}$. Assume that if a word to be accessed is in level 1, then the processor accesses it directly. If it is in level 2, then the word is first transferred to level 1 and then accessed by the processor. For simplicity, we ignore the time required for the processor to determine whether the word is in level 1 or level 2. Figure 4.2 shows the general shape of the curve that covers this situation. The figure shows the average access time to a two-level memory as a function of the hit ratio H , where H is defined as the fraction of all memory accesses that are found in the faster memory (e.g., the cache), T_1 is the access time to level 1, and T_2 is the access time to level 2.¹ As can be seen, for high percentages of level 1 access, the average total access time is much closer to that of level 1 than that of level 2.

In our example, suppose 95% of the memory accesses are found in the cache. Then the average time to access a word can be expressed as

$$(0.95)(0.01 \mu\text{s}) + (0.05)(0.01 \mu\text{s} + 0.1 \mu\text{s}) = 0.0095 + 0.0055 = 0.015 \mu\text{s}$$

The average access time is much closer to $0.01 \mu\text{s}$ than to $0.1 \mu\text{s}$, as desired.

Performance of two-level memory



$$T = T_1 H + (T_1 + T_2)(1 - H)$$

T : average (system) access time

T_1 : access time of M1 (e.g., cache, disk cache)

T_2 : access time of M2 (e.g., main memory, disk)

H : hit ratio (fraction of time reference is found in M1)

Performance of two-level memory

T : average (system) access time

T_1 : access time of M1 (e.g. cache)

T_2 : access time of M2 (e.g., main memory)

T_3 : access time of M3 (e.g., disk)

H_1 : hit ratio (fraction of time reference is found in M1)

H_2 : hit ratio (fraction of time reference is found in M2)

For main memory and disk

$$T_{\text{avg2}} = H_2 T_2 + (1 - H_2) (T_2 + T_3)$$

For cache

$$\begin{aligned} T = T_{\text{avg1}} &= H_1 T_1 + (1 - H_1) (T_1 + T_{\text{avg2}}) \\ &= T_1 + (1 - H_1) [T_2 + (1 - H_2) T_3] \end{aligned}$$

Locality of Reference

Memory references tend to cluster: 内存访问倾向于聚集

- Long time, clusters change
- Short time, fixed clusters of memory references(在短时间内，内存访问集中在固定的几个区域), 其原因:

1. Sequential execution
2. Narrow window of procedure depth
3. Repeated instructions
4. Data structures

- Spatial locality & Temporal locality
 - Memory locations clustered, eg, sequential access, data structures
 - memory locations recently used, eg, iteration loop

Locality Example

- **Spatial locality**

Memory locations clustered, eg, sequential access, data structures

```
for (i=0;i<10;i++) b[i]=0
```

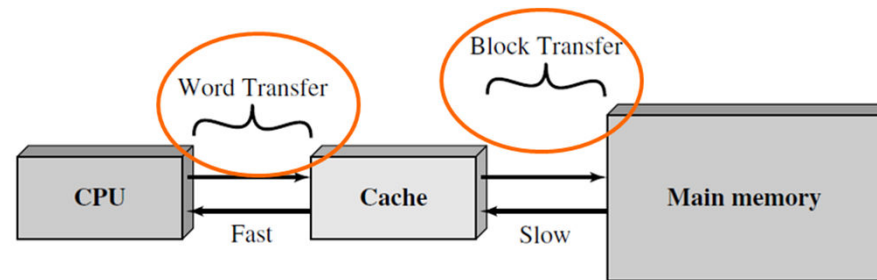
- **Temporal locality**

Memory locations recently used, eg, iteration loop

```
for (i=0;i<10;i++) a=a+1
```

4.2 Cache memory principles

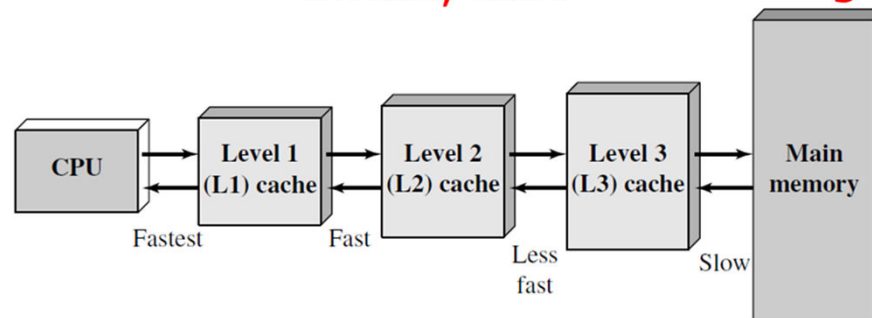
- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module



(a) Single cache

Small, fast

Large, slow



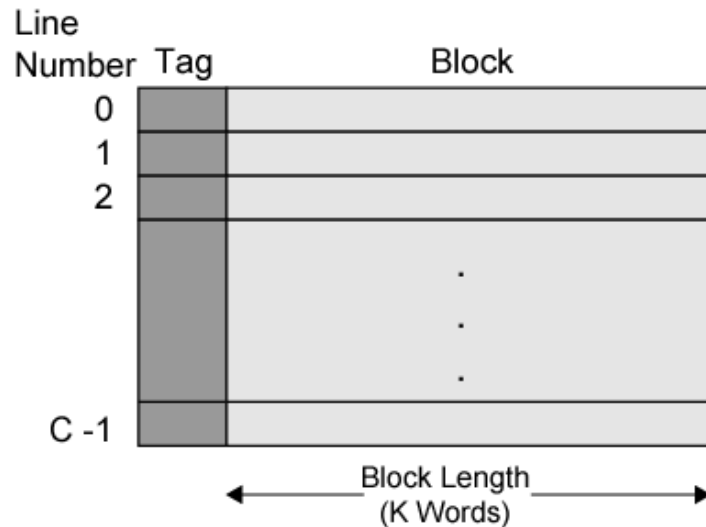
(b) Three-level cache organization



Cache

- **Copy** of portions of Main memory;
 - **Coherency** with Main memory.
-
- When a processor attempts to read a word of memory, check cache first:
 1. The word is in the cache: Plan A
 2. The word is not in the cache: Plan B

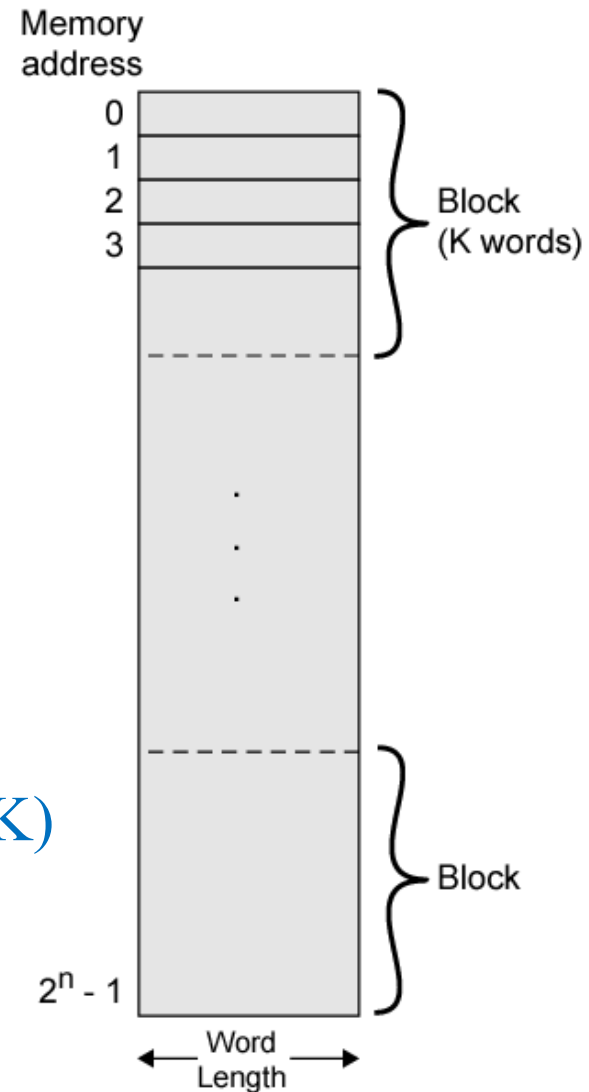
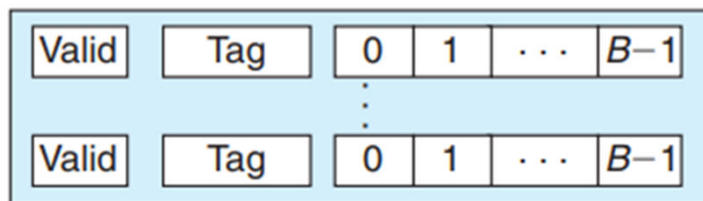
Cache/Main Memory Structure



(a) Cache

1 Cache line hold 1 Memory Block

C much less than Block number ($M = 2^n/K$)



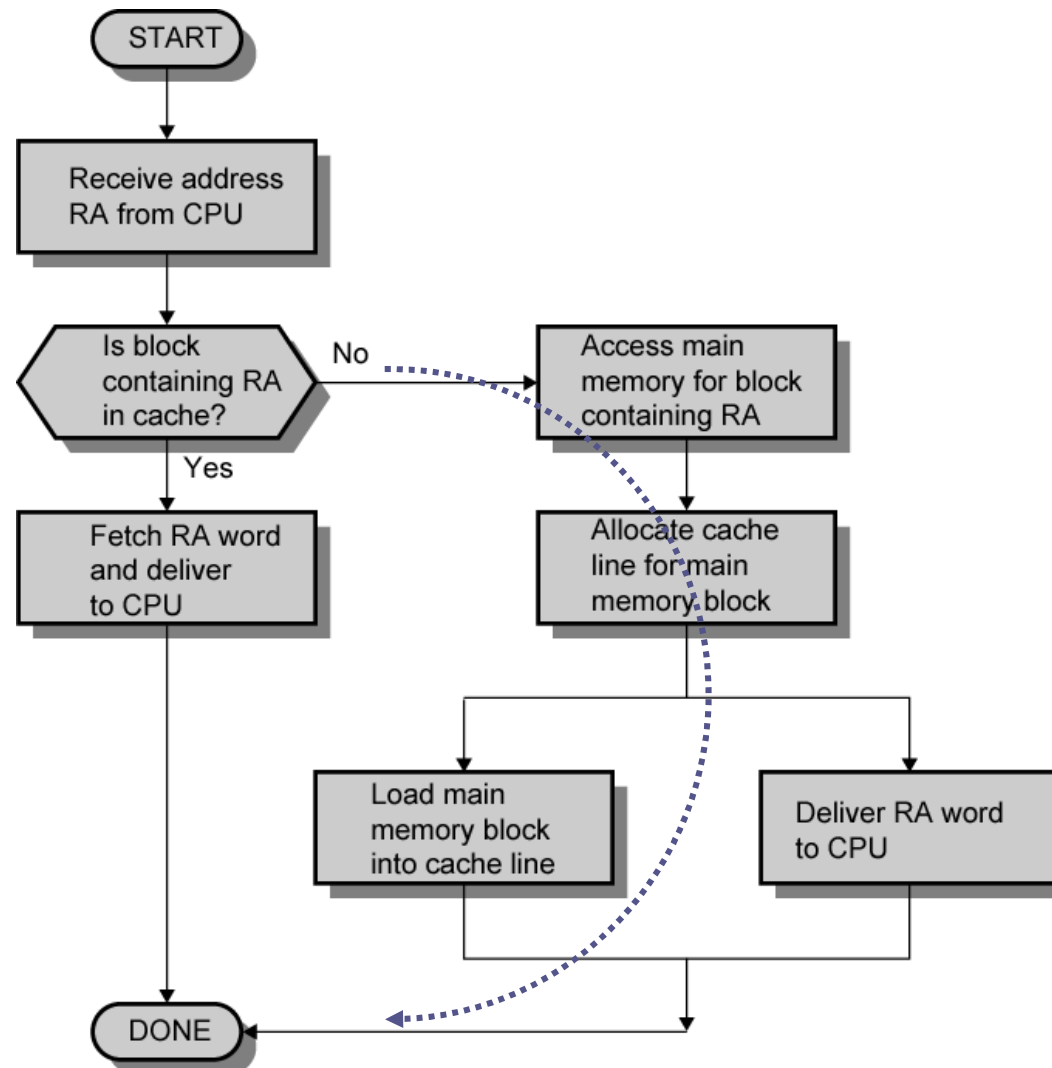
(b) Main memory



Cache operation - overview

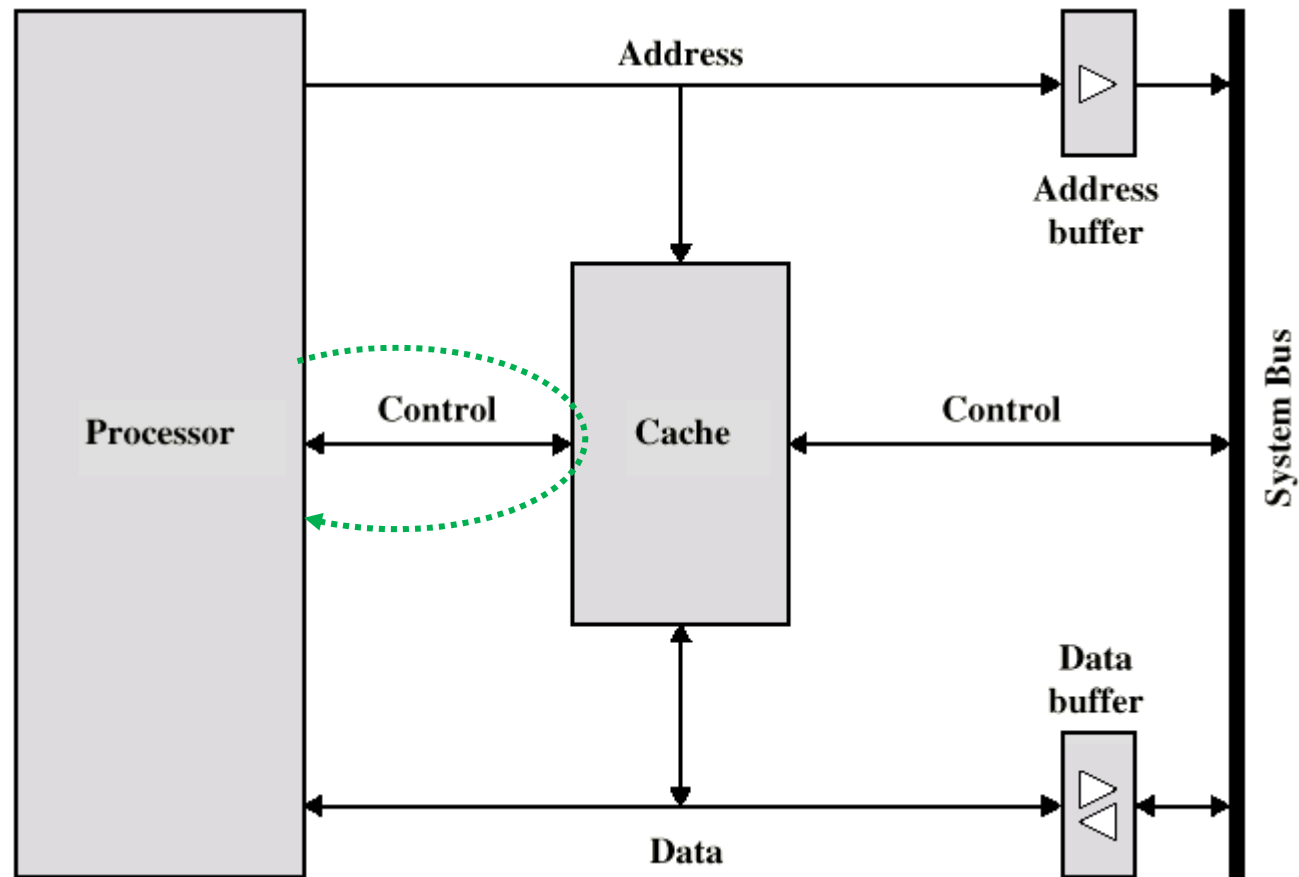
- CPU requests contents of memory location
- Check cache for this data
 - If present, get from cache (**hit**, fast)
 - If not present (**miss**), read required block from main memory to cache, then deliver from cache to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

Cache Read Operation - Flowchart

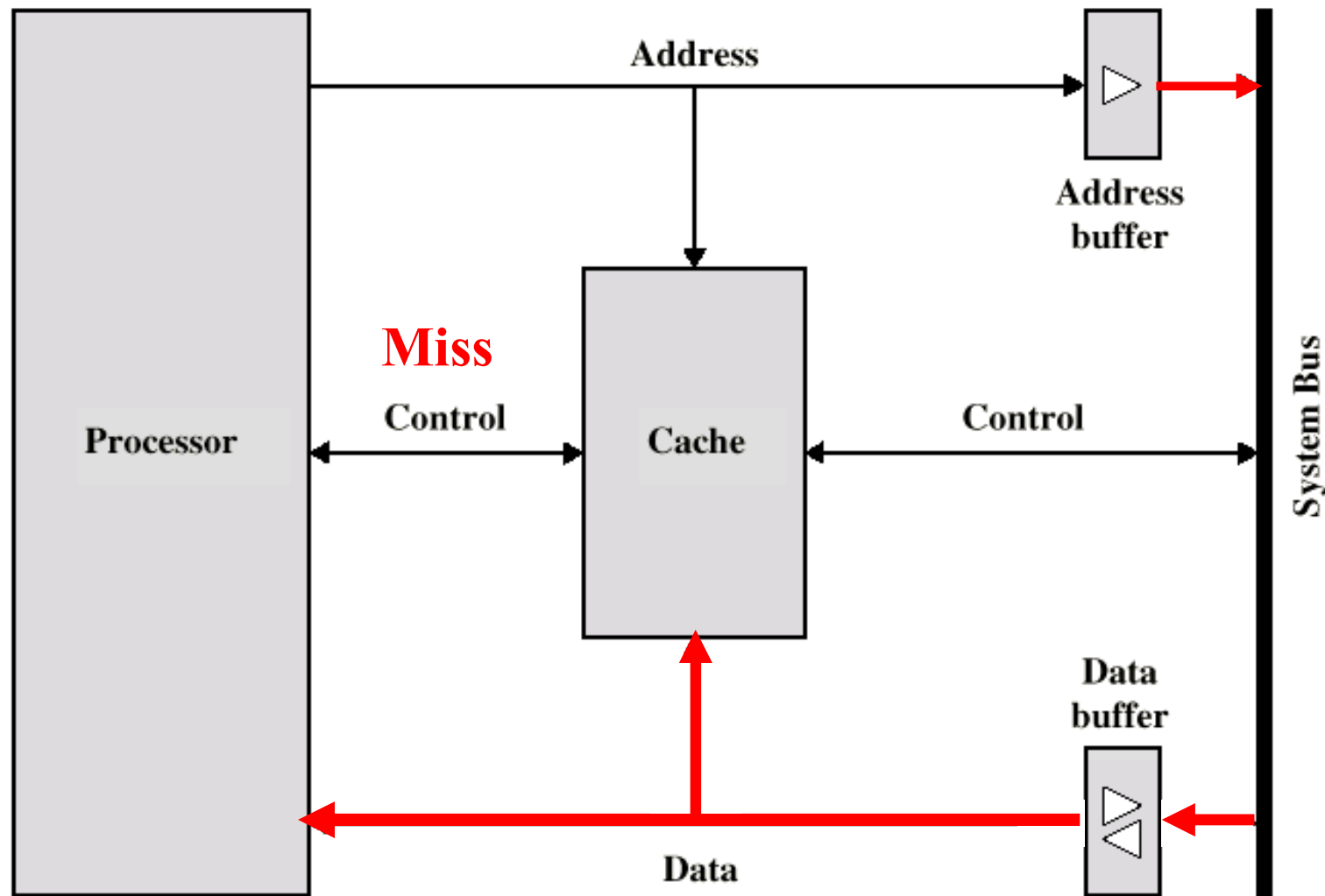


Parallel
operation

Typical Cache Organization



Typical Cache Organization



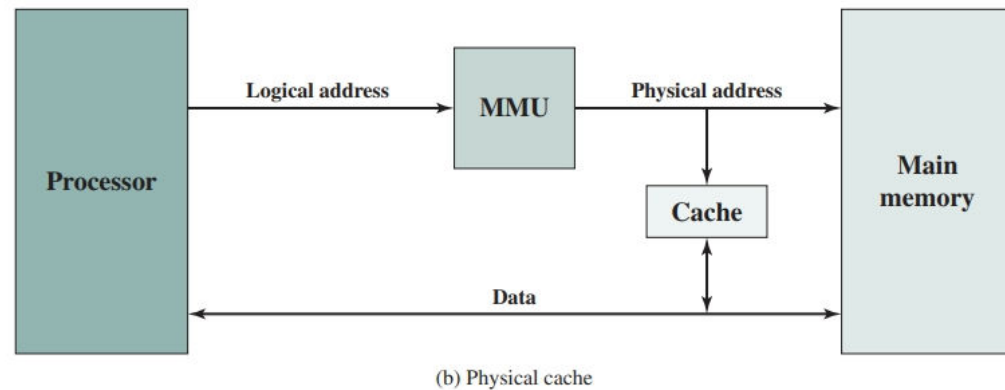
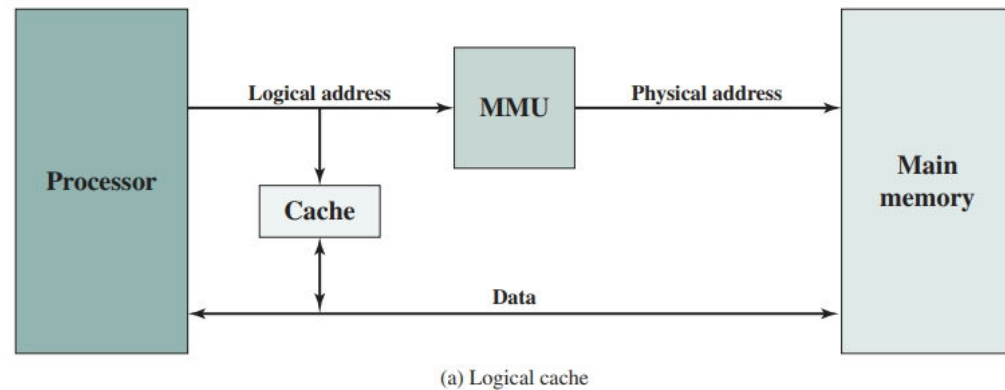


4.3 Elements of Cache Design

- Cache addresses
- Size
- Mapping Function
- Replacement Algorithm
- Write Policy
- Block Size
- Number of Caches

Logical Cache and Physical Cache

- Logical cache is faster than physical cache.
- It needs a **complete flush** with each context switch, or needs **pid bits**.



Size does matter

- Cost
 - More cache is expensive
- Speed
 - More cache is faster
 - Large caches tend to be slightly slower than small ones.
 - Chip and board area limits cache size.

IBM POWER6	PC/server	2007	64 kB/64 kB	4 MB	32 MB
IBM z10	Mainframe	2008	64 kB/128 kB	3 MB	24–48 MB
Intel Core i7 EE 990	Workstation/ server	2011	6 × 32 kB/ 32 kB	1.5 MB	12 MB
IBM zEnterprise 196	Mainframe/ server	2011	24 × 64 kB/ 128 kB	24 × 1.5 MB	24 MB L3 192 MB L4



Mapping Function

- Direct
- Associative
- Set associative

Mapping example

- Cache of 64kByte
- Cache block of 4 bytes
 - i.e. cache is 16k (2^{14}) lines of 4 bytes
- 16MBytes main memory
- 24 bit address
 - ($2^{24}=16\text{M}$)

Direct Mapping

- Each block of main memory maps to only one cache line
 - i.e. if a block is in cache, it must be in one specific place
- Address is in two parts:
 - Least Significant w bits identify unique word or byte
 - Most Significant s bits specify one memory block
 - The s MSBs are split into a cache line field r and a tag of $s-r$ (MSB $s-r$)

Direct Mapping function

$$i = j \bmod m$$

i = line index in cache

j = block index in main memory

m = number of lines of the cache
(Cache Size in Line)

$$j = \left\lfloor x / K \right\rfloor$$

x = Address in main memory
 K = block size in words
 w = bits for words in a block

$$j = x \gg w \quad (\text{Shift Right})$$

Direct Mapping

Address Structure

Tag s-r	Line or Slot r	Word w
8	14	2

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
 - 8 bit tag (=22-14)
 - 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

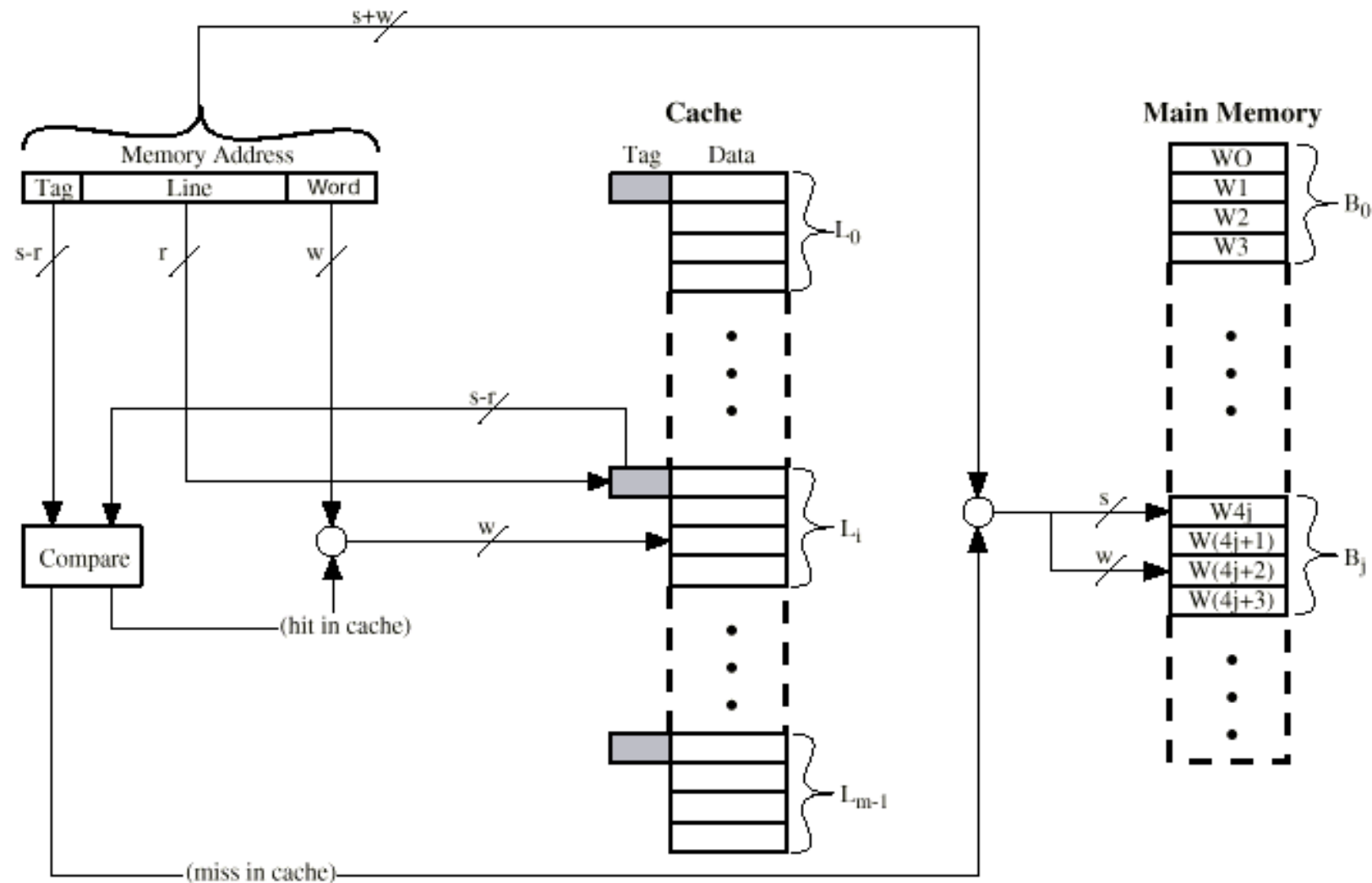
C662E3:

110001100110001011100011

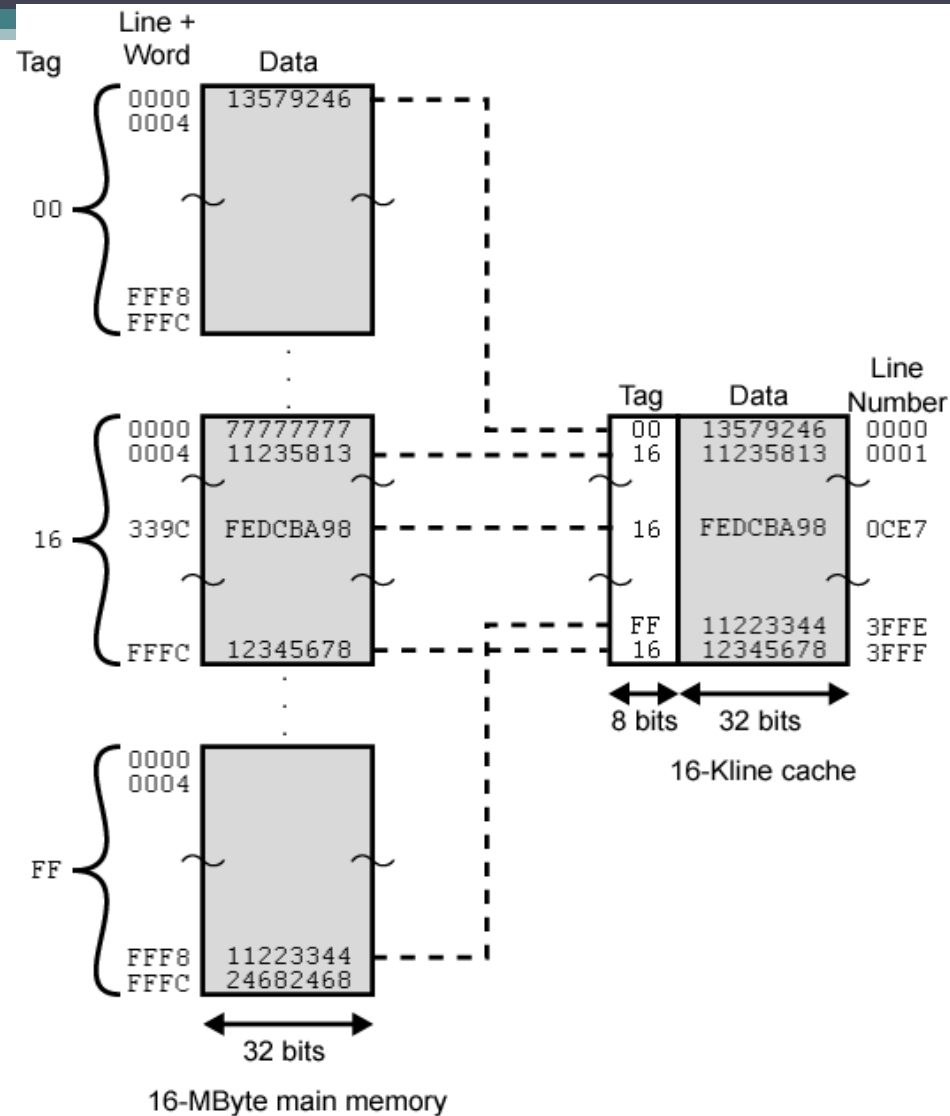
Direct Mapping Cache Line Table

- Cache line Main Memory blocks held
- 0 0, m, 2m, 3m... $2^s - m$
- 1 1, m+1, 2m+1... $2^s - m + 1$
- m-1 m-1, 2m-1, 3m-1... $2^s - 1$

Direct Mapping Cache Organization



Direct Mapping Example



	Tag	Line	Word
Main memory address =	8	14	2